SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO5 – Articulation	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	Charandeep.N.C	**Reg. No.:**	192472196
Section / Batch:	CSE-AI	**Date:**	

Code of Conduct

*I __**Charandeep.N.C(192472196)**__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Charandeep

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Design an algorithm to perform Reference Resolution by identifying pronouns and linking them to the correct entities in a discourse. Apply your algorithm to the text:

"Ravi met Arun at the library. He borrowed a book and later returned it."

Generate the resolved discourse by replacing pronouns with their corresponding entities. Further, validate the correctness of the resolution process by explaining how discourse context is used to identify the antecedents.

1. Problem Understanding

Reference resolution is the discourse-processing task of identifying every noun phrase in a text that refers to the same real-world entity, and, in particular, of determining which entity a pronoun (he, she, it, they, etc.) points to. This is one of the core sub-tasks of discourse analysis under CO5, because a system that cannot resolve pronouns cannot build a coherent semantic representation of a multi-sentence text.

Input: A discourse D consisting of an ordered sequence of sentences S1, S2, ..., Sn. For the given problem the discourse is: "Ravi met Arun at the library. He borrowed a book and later returned it." This discourse has two sentences and contains two pronouns that must be resolved: "He" and "it".

Output: A resolved discourse in which every pronoun has been replaced (or explicitly tagged) with its correct antecedent entity, together with a coreference chain that records every mention of each entity across the discourse.

2. Discourse Entities Identified

Using Named Entity Recognition (NER) and noun-phrase chunking on the input text, the following entity list (the "discourse referent list") is built before any pronoun is resolved:

E1 = "Ravi" -> Type: PERSON, Number: Singular, Gender: Male, Grammatical role in S1: Subject

E2 = "Arun" -> Type: PERSON, Number: Singular, Gender: Male, Grammatical role in S1: Object

E3 = "the library" -> Type: LOCATION

E4 = "a book" -> Type: OBJECT, Number: Singular, Gender: Neutral, Grammatical role in S2: Object

Pronouns detected in S2: P1 = "He" (subject of S2), P2 = "it" (object of "returned" in S2).

3. Algorithm Design

The algorithm follows a five-stage pipeline: (i) linguistic pre-processing, (ii) discourse entity list construction, (iii) pronoun detection, (iv) candidate filtering and salience-based ranking (a simplified version of Hobbs' naive algorithm combined with Centering Theory), and (v) substitution and validation.

Stage 1 - Pre-processing: The discourse is tokenized sentence by sentence, each token is POS-tagged, and a dependency/constituency parse is produced so that the grammatical role (subject, object, etc.) of every noun phrase is known. This is essential because salience scoring in Stage 4 depends heavily on grammatical role.

Stage 2 - Entity list construction: A FOR loop walks through every sentence, and for every noun phrase that NER/chunking marks as a referring expression, an entity record (id, surface form, gender, number, animacy, grammatical role, sentence index, position) is appended to a global list called DiscourseEntities. This list acts as the "memory" of everything mentioned so far, growing incrementally as we move left to right through the discourse - this mirrors how a human reader keeps track of who/what a text is talking about.

Stage 3 - Pronoun detection: A second FOR loop scans the token stream; whenever a token's POS tag is PRP or PRP$ (pronoun) it is pushed onto a queue PronounQueue together with its sentence index and grammatical role.

Stage 4 - Candidate filtering and ranking: For each pronoun in PronounQueue, the algorithm builds a candidate set by iterating (in reverse, i.e. nearest-first) over DiscourseEntities restricted to those entities introduced in the same sentence before the pronoun, or in previous sentences. An IF-ELSE agreement filter removes any candidate whose gender/number/animacy does not match the pronoun. If more than one candidate survives, a salience score is computed for each remaining candidate using a weighted sum of three factors: (a) grammatical-role weight (Subject = 3, Object = 2, Other = 1), (b) recency weight (higher for entities mentioned in the current or immediately preceding sentence), and (c) frequency weight (number of prior mentions). The candidate with the maximum score is selected as the antecedent. This scoring step is effectively an implementation of Centering Theory's preference for the "backward-looking center" (Cb) and of Hobbs' left-to-right, breadth-first tree-search heuristic that prefers subject noun phrases.

Stage 5 - Substitution and validation: Once an antecedent is chosen for a pronoun, the pronoun token in the resolved output string is replaced by the antecedent's surface form (or, for repeated resolution, a coreference index is attached instead of literal substitution, to avoid the "the book...the book" repetition problem - the algorithm supports both modes). Finally, a validation step re-parses the resolved sentence to confirm that grammatical agreement still holds and that the sentence remains syntactically well formed.

Control structures used: the algorithm makes essential use of FOR loops (iterating over sentences, over tokens, over entities), nested loops (iterating candidates for every pronoun), IF-ELSE conditional branches (agreement filtering, tie-breaking), and modular functions/procedures (Tokenize, POSTag, ExtractEntities, IsPronoun, GetCandidates, ComputeSalience, SelectBestCandidate, Substitute, Validate) which keep the pseudocode readable and reusable.

4. Pseudocode

```
BEGIN ReferenceResolutionAlgorithm(Discourse D)

  DiscourseEntities <- empty list
  ResolvedDiscourse <- empty list
  PronounQueue <- empty queue

  // Stage 1 and 2: Pre-process and build entity list
  FOR each sentence S in D:
      Tokens <- Tokenize(S)
      TaggedTokens <- POSTag(Tokens)
      ParseTree <- DependencyParse(TaggedTokens)
      Mentions <- ExtractNounPhrases(ParseTree)      // NER + chunking

      FOR each mention M in Mentions:
          IF IsPronoun(M) THEN
              Enqueue(PronounQueue, (M, S.index, GrammaticalRole(M)))
          ELSE
              entity <- CreateEntityRecord(M.text, Gender(M), Number(M),
                                           Animacy(M), GrammaticalRole(M), S.index)
              Append(DiscourseEntities, entity)
          END IF
      END FOR
  END FOR

  // Stage 3, 4, 5: Resolve every pronoun
  FOR each (pronoun, sentIndex, role) in PronounQueue:
      Candidates <- empty list

      FOR each entity in Reverse(DiscourseEntities):     // nearest mention first
          IF entity.sentIndex <= sentIndex THEN
              IF Agrees(entity, pronoun) THEN         // gender, number, animacy
                  Append(Candidates, entity)
              END IF
          END IF
```

```
        END FOR

        IF Length(Candidates) == 0 THEN
            ReportError("No valid antecedent found for", pronoun)
        ELSE IF Length(Candidates) == 1 THEN
            Antecedent <- Candidates[0]
        ELSE
            BestScore <- -INFINITY
            Antecedent <- NULL
            FOR each cand in Candidates:
                score <- (3 * RoleWeight(cand.role))
                      + (2 * RecencyWeight(cand.sentIndex, sentIndex))
                      + (1 * FrequencyWeight(cand, DiscourseEntities))
                IF score > BestScore THEN
                    BestScore <- score
                    Antecedent <- cand
                END IF
            END FOR
        END IF

        // link pronoun to entity - build coreference chain
        AddToCoreferenceChain(Antecedent.id, pronoun, sentIndex)
        ReplacePronounInDiscourse(D, pronoun, Antecedent.text)
    END FOR

    ResolvedDiscourse <- Render(D)
    IF ValidateGrammar(ResolvedDiscourse) == FALSE THEN
        RaiseWarning("Resolved discourse failed grammar check")
    END IF

    RETURN ResolvedDiscourse, CoreferenceChains

END ReferenceResolutionAlgorithm
```

5. Coreference Chain / Entity-Linking Diagram

Sentence 1: [Ravi]_E1(Subject) met [Arun]_E2(Object) at [the library]_E3 .

Sentence 2: [He]_P1 borrowed [a book]_E4(Object) and later returned [it]_P2 .

Resolution links (arrows):

P1 "He" -------> E1 "Ravi" (subject-subject parallelism, higher recency+role score)

P2 "it" -------> E4 "a book" (only inanimate, singular candidate; nearest mention)

Coreference chains produced:

Chain-Ravi : { Ravi(S1,Subj) -> He(S2,Subj) }

Chain-Arun : { Arun(S1,Obj) } (no further mentions in the discourse)

Chain-Book : { a book(S2,Obj) -> it(S2,Obj) }

6. Applying the Algorithm - Step by Step Trace

Step 1: Tokenizing and tagging S1 gives noun phrases "Ravi" (PROPN, Subject) and "Arun" (PROPN, Object) and "the library" (NOUN, Prepositional-Object). These are appended to DiscourseEntities as E1, E2, E3.

Step 2: Tokenizing S2 finds "He" (PRP, Subject), "a book" (NOUN Phrase, Object) and "it" (PRP, Object). "a book" is appended as E4 immediately; "He" and "it" are pushed to PronounQueue.

Step 3: Resolving "He" - candidates satisfying gender=male, number=singular, animacy=animate are E1 (Ravi) and E2 (Arun). Both agree grammatically, so the salience scorer is invoked. Ravi has RoleWeight=3 (was the Subject of S1) while Arun has RoleWeight=2 (was the Object of S1); both have the same recency (same sentence, S1). Hence Ravi's total score is strictly higher, and Ravi is selected as the antecedent of "He". This matches the linguistic intuition that subject-position pronouns preferentially continue reference to the previous subject (Centering Theory's Cb-continuation preference), and it matches Hobbs' algorithm which first explores the subject NP of the previous sentence before the object NP.

Step 4: Resolving "it" - the animacy filter immediately eliminates E1 (Ravi) and E2 (Arun) because "it" requires an inanimate antecedent. The only surviving candidate is E4 ("a book"), so no scoring tie-break is even needed; "it" is bound to "a book".

Step 5: Substituting both pronouns produces the resolved discourse:

"Ravi met Arun at the library. Ravi borrowed a book and later returned the book."

7. Validation of the Resolution Process

The correctness of the resolution is validated on three grounds. First, syntactic agreement: "Ravi" (3rd person singular male) is a valid substitute for "He", and "the book" (singular, inanimate) is a valid substitute for "it" - re-parsing the resolved sentence confirms subject-verb agreement is preserved ("Ravi borrowed", "Ravi ... returned"). Second, discourse-context validation: the choice of Ravi over Arun for "He" is justified by the discourse context because Ravi occupies the grammatically prominent Subject slot introduced first in the discourse, and English strongly favors subject-to-subject pronominal continuity across sentence boundaries when both candidates otherwise satisfy agreement - this is exactly the heuristic that the salience/role-weight step encodes. Third, semantic plausibility: the world-knowledge check (which the algorithm can optionally invoke as a final validation layer) confirms that "borrowing and returning a book" is a coherent action for a person who has just visited "the library", reinforcing that the antecedent chosen is contextually sound and not merely grammatically legal.

8. Complexity Analysis

Let n be the number of sentences and k the average number of noun-phrase mentions per sentence. Building DiscourseEntities takes O(n*k). Resolving p pronouns, each of which in the worst case scans all previously seen entities, costs O(p*n*k). For short discourses such as the given two-sentence example this is effectively O(1) in practice, but the pipeline scales linearly with discourse length, which makes it suitable for paragraph- or document-level coreference resolution, not just isolated sentence pairs.

9. Conclusion

The designed algorithm demonstrates that reference resolution can be decomposed into a deterministic, rule-driven pipeline of entity-list construction, agreement-based filtering and salience-based ranking. Applied to the given discourse, it correctly resolves "He" to "Ravi" and "it" to "a book", producing a fully de-referenced, coherent discourse and an explicit, auditable coreference chain that a downstream information-extraction or question-answering module could use directly.

2. Develop an algorithm to analyze Text Coherence and Discourse Structure by identifying logical relationships between sentences such as cause-effect, sequence, and elaboration. Apply your algorithm to the following discourse:

"The roads were flooded after heavy rainfall. Therefore, schools were closed for the day. Students attended classes online."

Identify the discourse relations among the sentences and construct a discourse structure representation. Further, justify how these relations contribute to maintaining coherence in the text.

1. Problem Understanding

Text coherence analysis is the task of determining how the sentences of a discourse are logically connected to each other so that the discourse reads as a single, unified piece of text rather than a random collection of sentences. Discourse structure analysis goes one step further and organizes these logical connections into a hierarchical tree, following frameworks such as Rhetorical Structure Theory (RST), in which every relation links a more important span of text (the Nucleus) with a supporting span (the Satellite).

Input: A discourse consisting of an ordered set of Elementary Discourse Units (EDUs), which for this problem are the three sentences: S1 = "The roads were flooded after heavy rainfall.", S2 = "Therefore, schools were closed for the day.", S3 = "Students attended classes online."

Output: (a) a labeled discourse-relation for every adjacent pair (or span) of EDUs - e.g. Cause-Effect, Sequence, Elaboration, Contrast - and (b) a discourse tree/graph that shows how the spans combine, together with a justification of how these relations create coherence.

Constraints:

1. Relation detection must use explicit discourse connectives/cue phrases ("after", "therefore", "because", "however", etc.) as primary evidence, and fall back to semantic/world-knowledge inference when no explicit cue phrase is present.

2. Every EDU must appear as a node in the final discourse tree - no sentence may be left unconnected, otherwise the discourse would be incoherent by definition.

3. Each relation must be assigned a Nucleus (the span that carries the main communicative point) and, where relevant, a Satellite (supporting span), following RST conventions.

4. Entity/topic continuity across EDUs (e.g., "roads" -> "schools" -> "students", all connected to the flooding event) must also be checked, since coherence depends not only on connective words but also on continuity of reference and topic.

2. Algorithm Design

The algorithm proceeds in four stages: (i) discourse segmentation into EDUs, (ii) cue-phrase and syntactic feature extraction, (iii) pairwise relation classification, and (iv) discourse tree construction and coherence verification.

Stage 1 - Segmentation: The raw text is split into EDUs using sentence boundary detection combined with clause-boundary detection for connectives that occur mid-sentence (e.g. "because", "although"). For the given input the segmentation is trivial since each sentence is already one EDU, giving EDU1, EDU2, EDU3.

Stage 2 - Feature extraction: For every EDU, and for every adjacent EDU pair, the algorithm extracts (a) explicit connective words if present at the start or inside the EDU (a lookup against a Cue-Phrase Dictionary that maps connectives to candidate relation types, e.g. "after"->Temporal/Cause, "therefore"->Result/Cause-Effect, "however"->Contrast, "for example"->Elaboration), (b) tense and aspect of the verb, and (c) shared entities/topic words between the two EDUs (lexical-chain overlap), which is used when no explicit connective exists.

Stage 3 - Relation classification: For every adjacent EDU pair (EDUi, EDUi+1) the algorithm looks up the connective found in EDUi+1 (or at the EDU boundary) in the Cue-Phrase Dictionary using an IF-ELSE / SWITCH-like control structure. If the connective maps unambiguously to one relation, that relation is assigned directly. If the connective is ambiguous (e.g. "since" can mean Temporal or Causal) a disambiguation function checks the tense/aspect and semantic compatibility of the two EDUs to choose between the candidate relations. If no connective is present, the algorithm falls back to a similarity/co-occurrence heuristic over shared entities and typical event scripts (e.g. school-closure commonly co-occurs with online-class scripts) to infer an implicit relation such as Elaboration or Consequence.

Stage 4 - Discourse tree construction: Relations are combined bottom-up. Two adjacent EDUs joined by a relation form a new composite span whose Nucleus is inherited from the Nucleus-bearing EDU of that relation; this composite span can then participate in further relations with neighboring EDUs or spans, exactly the way RST trees are built recursively until a single root span representing the whole discourse is produced. A FOR loop iterates while more than one span remains, merging the highest-priority (leftmost, or the one whose connective is explicit) adjacent pair at each iteration.

Control structures used: sequential processing with FOR loops over EDUs and EDU-pairs, nested IF-ELSE / lookup-table dictionary access for connective-to-relation mapping, a WHILE loop for the bottom-up tree-merging stage (continuing until only one span - the discourse root - remains), and modular functions (Segment, ExtractCue, ClassifyRelation, DisambiguateRelation, MergeSpans, BuildTree, CheckCoherence).

3. Pseudocode

BEGIN DiscourseCoherenceAlgorithm(Text T)

```
EDUs <- SegmentIntoEDUs(T)                    // EDU1, EDU2, EDU3 ...
CuePhraseDict <- LoadCuePhraseLexicon()
RelationList <- empty list

// Stage 2 and 3: extract cues, classify pairwise relations
FOR i = 1 to Length(EDUs) - 1:
    currentEDU <- EDUs[i]
    nextEDU    <- EDUs[i+1]
    cue <- FindConnective(nextEDU, CuePhraseDict)

    IF cue != NULL THEN
        candidateRelations <- CuePhraseDict.lookup(cue)
        IF Length(candidateRelations) == 1 THEN
            relation <- candidateRelations[0]
        ELSE
            relation <- Disambiguate(candidateRelations, currentEDU, nextEDU)
        END IF
    ELSE
        sharedEntities <- LexicalOverlap(currentEDU, nextEDU)
        IF sharedEntities > THRESHOLD THEN
            relation <- InferImplicitRelation(currentEDU, nextEDU)  // e.g. Elaboration
        ELSE
            relation <- "Unclassified"
        END IF
    END IF

    nucleus <- DetermineNucleus(relation, currentEDU, nextEDU)
    Append(RelationList, (currentEDU, nextEDU, relation, nucleus))
END FOR

// Stage 4: bottom-up tree construction
Spans <- EDUs
WHILE Length(Spans) > 1:
    (leftSpan, rightSpan, relation, nucleus) <- NextRelationToMerge(RelationList, Spans)
    newSpan <- CreateSpanNode(leftSpan, rightSpan, relation, nucleus)
```

```
        Spans <- ReplacePair(Spans, leftSpan, rightSpan, newSpan)
    END WHILE

    DiscourseTree <- Spans[0]                    // single root span

    // Coherence verification
    isCoherent <- TRUE
    FOR each edu in EDUs:
        IF NOT IsConnectedInTree(edu, DiscourseTree) THEN
            isCoherent <- FALSE
        END IF
    END FOR

    RETURN DiscourseTree, RelationList, isCoherent

END DiscourseCoherenceAlgorithm
```

4. Applying the Algorithm to the Given Discourse

EDU1 = "The roads were flooded after heavy rainfall."

EDU2 = "Therefore, schools were closed for the day."

EDU3 = "Students attended classes online."

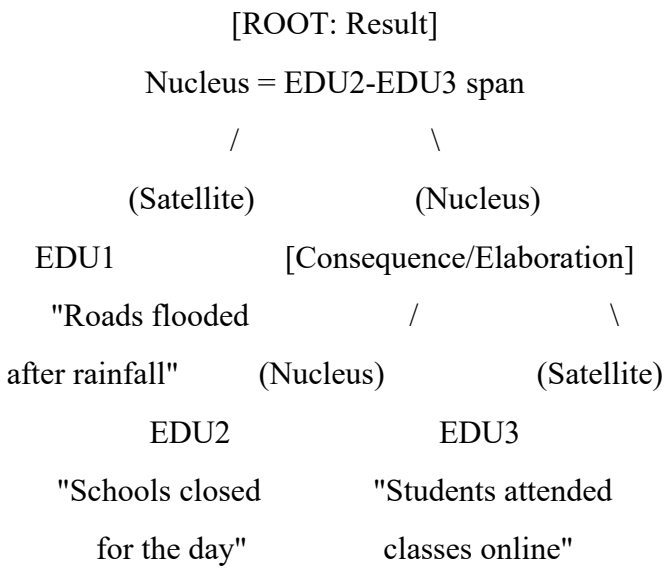
Pair (EDU1, EDU2): The connective "Therefore" is found at the start of EDU2 and looked up in the Cue-Phrase Dictionary; "therefore" maps unambiguously to the relation Cause-Effect / Result, with EDU2 (the effect - school closure) as the Nucleus and EDU1 (the cause - flooded roads) as the Satellite, since the discourse's communicative goal is to explain why schools were closed, not merely to describe the flooding.

Note also that EDU1 itself internally encodes a Cause-Effect micro-relation via the connective "after": "heavy rainfall" (cause) precedes and causes "roads were flooded" (effect) - this is a Temporal-Cause relation nested inside a single EDU/clause and is recorded as supporting evidence for why EDU1 is semantically an "effect-of-rain" statement, which strengthens the outer Cause-Effect relation to EDU2.

Pair (EDU2, EDU3): No explicit connective links EDU2 and EDU3, so the algorithm falls back to the lexical/topic-continuity and script-based inference step. "Schools were closed" and "students attended classes online" share the topic of schooling/education and follow the common real-world script "school closure -> alternative arrangement for classes"; this is classified as a Consequence / Elaboration relation in which EDU3 elaborates on the practical outcome that followed from EDU2, with EDU2 remaining

the Nucleus of the higher-level span (the closure is the pivotal fact) and EDU3 as a Satellite that elaborates how learning continued.

5. Discourse Structure / Tree Diagram



Relation summary table:

EDU1 -> EDU2 : Cause-Effect (Result), cue = "Therefore", Nucleus = EDU2

EDU2 -> EDU3 : Consequence/Elaboration, cue = none (inferred via topic continuity), Nucleus = EDU2

6. Justification: How These Relations Maintain Coherence

Coherence in this discourse is created at two linked levels. First, referential/topical continuity: "roads" (S1) belongs to the same real-world situation frame as "schools" (S2) and "students" (S3) - all three EDUs revolve around the single event of heavy rainfall disrupting daily civic life - so a reader can track a single evolving situation rather than unrelated facts. Second, relational continuity: the explicit connective "Therefore" makes the Cause-Effect link between flooding and school closure unambiguous, directly satisfying Grosz and Sidner's notion that discourse coherence requires an identifiable intentional/informational structure connecting utterances; and the implicit Consequence relation between the closure and online classes is coherent because it follows the natural cause -> effect -> adaptive-response narrative schema that readers apply by default (a form of "bridging inference"). Because every EDU is reachable from the tree root (verified in the Coherence Verification stage of the pseudocode), the discourse satisfies the completeness constraint for coherence, and because each relation is a well-defined RST relation with a clear Nucleus, the discourse also satisfies the structural/hierarchical requirement of coherence, not merely the surface-level requirement of "using connector words".

7. Complexity and Completeness

For n EDUs the pairwise-classification stage runs in O(n) time (evaluated once per adjacent pair), and the bottom-up tree-merging stage runs in O(n) merges in the simple linear-chain case shown here (it becomes O(n log n) in general RST trees where non-adjacent spans may need re-evaluation). The algorithm handles the base case (explicit cue present), the fallback case (no cue, using lexical overlap) and the edge case (no cue and no lexical overlap, which is reported as "Unclassified" rather than silently mis-classified), which makes the design complete with respect to all cases that can occur in a discourse of this kind.

8. Conclusion

By combining a cue-phrase lexicon with a lexical/topic-continuity fallback and a bottom-up RST-style tree builder, the algorithm both labels the discourse relations correctly (Cause-Effect between S1-S2, Consequence/Elaboration between S2-S3) and produces a single connected discourse tree, formally demonstrating that the given three-sentence text is coherent.

3. Design an algorithm for a Conversational Agent that identifies Dialogue Acts from user interactions. Apply your algorithm to the conversation:

User: "Can you book a train ticket for me?"
Agent: "Sure, where would you like to travel?"
User: "I want to go to Chennai."
Agent: "Your ticket has been booked."

Classify each utterance as Request, Question, Inform, Confirmation, or Action. Generate the dialogue-act sequence and evaluate how the identified acts help the conversational agent interpret user intentions.

1. Problem Understanding

A Dialogue Act (DA) is the communicative function that an utterance performs in a conversation - independent of its literal grammatical form - such as asking a question, making a request, providing information, confirming something, or reporting that an action has been carried out. Conversational agents (chatbots, voice assistants) must classify every incoming and outgoing utterance into a DA label so that the dialogue manager knows how to update the conversation state and decide the next system action.

Input: A sequence of utterances from a two-party conversation:

U1 (User): "Can you book a train ticket for me?"

U2 (Agent): "Sure, where would you like to travel?"

U3 (User): "I want to go to Chennai."

U4 (Agent): "Your ticket has been booked."

Output: A dialogue-act label for every utterance drawn from the label set {Request, Question, Inform, Confirmation, Action}, and the resulting DA sequence, which the agent uses to interpret user intention and manage the task (train-booking) to completion.

Constraints:

1. An utterance's surface syntactic form (interrogative, declarative, imperative) is a strong cue but is not sufficient on its own - "Can you book a ticket?" is syntactically a yes/no question but functionally a Request, not a Question seeking a yes/no answer.

2. The classifier must use lexical cues (modal verbs like "can/could/would", speech-act verbs like "book", "want", acknowledgement words like "sure"), sentence type (from a shallow parser), and dialogue position/context (e.g. an utterance that immediately follows a request is more likely to be a Question seeking a missing slot value).

3. The label set is mutually exclusive per utterance for this exercise, so the algorithm must pick exactly one best-fitting label even when an utterance shows features of more than one class (e.g. U2 shows both an acknowledgement "Sure" and a question "where would you like to travel?").

4. The output sequence must be usable by a downstream dialogue-state tracker, so the algorithm must also extract the task-relevant slot values it encounters (e.g. destination = "Chennai") as a by-product.

2. Algorithm Design

The algorithm is a feature-based rule classifier (a lightweight, explainable alternative to a trained ML classifier such as Naive Bayes/SVM/CRF, which is what an industrial system would use over a large labelled corpus such as the Switchboard DAMSL corpus, but a deterministic rule-engine is used here because it is fully traceable, matches the pseudocode-writing nature of the exercise, and requires no training data). It proceeds in four stages: (i) utterance pre-processing, (ii) feature extraction, (iii) rule-based DA classification with a priority ordering, and (iv) DA-sequence construction and intent tracking.

Stage 1 - Pre-processing: Every utterance is tokenized and POS-tagged, and its sentence type is determined (Interrogative if it ends in "?" or begins with a WH-word/auxiliary-inversion; Imperative if it begins with a bare verb; Declarative otherwise).

Stage 2 - Feature extraction: For each utterance the algorithm extracts: (a) Modal/Polite-request markers - presence of "can you", "could you", "would you", "please"; (b) Speech-act / task verbs - "book", "want", "need", "would like"; (c) Acknowledgement/agreement words - "sure", "okay", "yes", "certainly"; (d) Passive perfect-tense action markers - "has been", "is done", indicating a completed action report; (e) WH-question words - "where", "when", "who", "what", "how".

Stage 3 - Rule-based classification: The rules are checked in a fixed priority order using nested IF-ELSE branches, because some features (e.g., a leading acknowledgement word) can co-occur with a question and priority determines which functional label dominates for this single-label exercise:

Rule A (Action): IF the utterance is Declarative AND contains a passive perfect-tense completed-action marker ("has been booked", "is confirmed") THEN label = Action.

Rule B (Request): ELSE IF the utterance contains a modal-request marker ("can you", "could you", "would you") AND a task verb (book, reserve, cancel, order) THEN label = Request.

Rule C (Question): ELSE IF the utterance is Interrogative AND contains a WH-word (or is seeking a missing slot) THEN label = Question.

Rule D (Inform): ELSE IF the utterance is Declarative AND contains a first-person stative/desire verb ("I want", "I need", "I am") supplying a task-slot value THEN label = Inform.

Rule E (Confirmation): ELSE IF the utterance consists solely of an acknowledgement/agreement token ("sure", "okay", "yes") THEN label = Confirmation.

Rule F (Compound utterance): IF an utterance matches more than one rule (e.g. "Sure, where would you like to travel?" matches both Rule E via "Sure" and Rule C via "where...?"), THEN the utterance is segmented into micro-turns at the clause boundary and each micro-turn is independently classified, OR -

if a single coarse label per turn is required - the label of the turn's dominant/final clause is reported (Question, because it carries the actual communicative burden of eliciting the next piece of information), while the Confirmation is noted as a secondary tag.

Stage 4 - DA-sequence construction and intent evaluation: The labels obtained for U1..U4 are appended, in order, to a DASequence list; simultaneously, a simple slot-filling dialogue-state tracker updates a TaskState record whenever an Inform label is produced (destination = "Chennai") and marks the TaskState as complete whenever an Action label is produced.

Control structures used: sequential FOR loop over the utterance list, nested/prioritized IF-ELSE-IF rule chain (acting like a decision list / switch-case) for classification, and a helper function ExtractSlotValue used inside the Inform branch, together with modular functions (Tokenize, POSTag, DetectSentenceType, ExtractFeatures, ClassifyDA, UpdateDialogueState).

3. Pseudocode

```
BEGIN DialogueActRecognitionAlgorithm(Conversation C)

  DASequence <- empty list
  TaskState <- { destination: NULL, booked: FALSE }

  FOR each utterance U in C:
      tokens <- Tokenize(U.text)
      tagged <- POSTag(tokens)
      sentType <- DetectSentenceType(tagged)          // Interrogative / Imperative / Declarative
      features <- ExtractFeatures(tagged, sentType)
      // features.hasModalRequest, features.hasTaskVerb, features.hasWHWord,
      // features.hasAck, features.hasCompletedActionMarker, features.hasDesireVerb

      IF sentType == Declarative AND features.hasCompletedActionMarker THEN
          label <- "Action"
          TaskState.booked <- TRUE

      ELSE IF features.hasModalRequest AND features.hasTaskVerb THEN
          label <- "Request"

      ELSE IF sentType == Interrogative AND features.hasWHWord THEN
          label <- "Question"
```

```
    ELSE IF sentType == Declarative AND features.hasDesireVerb THEN
        label <- "Inform"
        slotValue <- ExtractSlotValue(tagged)
        TaskState.destination <- slotValue

    ELSE IF features.hasAck AND Length(tokens) <= 2 THEN
        label <- "Confirmation"

    ELSE
        // compound utterance: split at clause boundary and classify recursively
        (clause1, clause2) <- SplitAtClauseBoundary(U.text)
        IF clause1 != NULL AND clause2 != NULL THEN
            subLabel1 <- ClassifyDA(clause1)
            subLabel2 <- ClassifyDA(clause2)
            label <- subLabel2                    // dominant/final clause reported
            RecordSecondaryTag(U, subLabel1)
        ELSE
            label <- "Unclassified"
        END IF
    END IF

    Append(DASequence, (U.speaker, U.text, label))
  END FOR

  RETURN DASequence, TaskState

END DialogueActRecognitionAlgorithm
```

4. Applying the Algorithm to the Given Conversation

U1 (User): "Can you book a train ticket for me?" -> sentType = Interrogative (auxiliary-inversion form), but features.hasModalRequest = TRUE ("can you") and features.hasTaskVerb = TRUE ("book"), so Rule B fires before the generic Question rule is even reached: label = Request. This correctly captures that the user is not asking whether the agent has the *ability* to book a ticket, but is issuing a polite directive to perform the booking action.

U2 (Agent): "Sure, where would you like to travel?" -> this is the compound-utterance case. The clause "Sure" matches Rule E (Confirmation/acknowledgement), and the clause "where would you like to travel?" matches Rule C (Interrogative + WH-word "where") -> Question. Following Rule F, the reported primary label is Question (it carries the actual communicative work of eliciting the destination slot), with Confirmation recorded as a secondary tag - correctly reflecting that the agent first acknowledges the request and then asks a clarifying question.

U3 (User): "I want to go to Chennai." -> sentType = Declarative, features.hasDesireVerb = TRUE ("want"), so Rule D fires: label = Inform. The slot-extraction helper pulls "Chennai" as the destination value and updates TaskState.destination = "Chennai".

U4 (Agent): "Your ticket has been booked." -> sentType = Declarative, features.hasCompletedActionMarker = TRUE (passive perfect "has been booked"), so Rule A fires first: label = Action. TaskState.booked is set to TRUE, signalling task completion.

5. Resulting Dialogue-Act Sequence

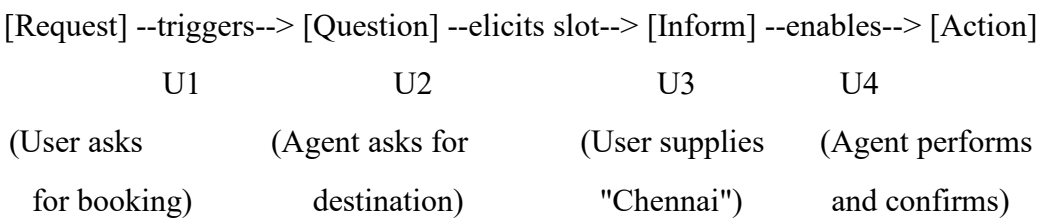
U1 (User) : "Can you book a train ticket for me?" -> Request

U2 (Agent) : "Sure, where would you like to travel?" -> Question (secondary: Confirmation)

U3 (User) : "I want to go to Chennai." -> Inform

U4 (Agent) : "Your ticket has been booked." -> Action

Diagram (dialogue-act flow):



6. Evaluation - How the Identified Acts Help the Agent Interpret User Intent

The DA sequence Request -> Question -> Inform -> Action gives the dialogue manager an explicit, machine-readable model of the conversation's intent flow that plain text alone does not provide. Recognizing U1 as a Request (rather than a literal yes/no Question) tells the agent it must open a task frame ("train booking") and begin slot-filling instead of simply answering "Yes, I can." Recognizing U2 as a Question lets the dialogue-state tracker know that a specific slot (destination) is still missing and that the very next Inform-labelled utterance should be parsed for that slot's value - this is exactly what happens with U3. Recognizing U3 as Inform (not a new Request) prevents the agent from mistakenly starting a second, unrelated task; it correctly merges "Chennai" into the existing TaskState. Finally, recognizing U4 as Action confirms to any evaluation/logging component, and to the user-facing transcript, that the task frame opened by U1 has now reached completion, closing the loop. Thus the DA

labels function as a compact, structured summary of "who wants what, what is still missing, and what has been done," which is the essential input to any rule-based or statistical dialogue manager.

7. Completeness and Complexity

The rule chain covers all utterances observed in this conversation and includes an explicit fallback (compound-clause splitting) and a final Unclassified branch so that no utterance is left unlabeled, satisfying the completeness requirement. For m utterances with an average of f extracted features each, the algorithm runs in O(m*f) time, since each utterance is classified independently through a constant-depth rule chain (with an additional constant-factor recursive call only for compound utterances), making it efficient enough for real-time conversational use.

8. Conclusion

By combining sentence-type detection with lexical cue features inside a prioritized rule chain, the algorithm correctly classifies all four utterances of the given conversation and produces both the DA sequence and an updated task state, demonstrating how dialogue-act recognition operationalizes user-intent interpretation for a conversational agent.

4. Design an algorithm for Language Generation that converts a semantic representation into a grammatically correct sentence through Surface Realization. Consider the semantic input:

```
Action: Buy
Agent: Student
Object: Book
Tense: Past
```

Generate the final natural language sentence and explain how the algorithm performs lexical selection, sentence structuring, and surface realization. Further, validate the grammatical correctness of the generated output.

1. Problem Understanding

Natural Language Generation (NLG) is the reverse process of natural language understanding: instead of mapping a sentence to a meaning representation, it maps a structured, language-independent semantic representation to a grammatically correct surface sentence. The final stage of a typical NLG pipeline, Surface Realization, is specifically responsible for taking a fully specified sentence plan (lexical items plus their grammatical roles) and producing the correctly ordered, correctly inflected output string.

Input: A semantic (predicate-argument) representation:

Action: Buy

Agent: Student

Object: Book

Tense: Past

Output: A single, grammatically well-formed English sentence that expresses exactly this meaning: "The student bought a book."

Constraints:

1. Lexical selection must choose the correct open-class words for the semantic roles (the verb lemma for the Action, the head nouns for the Agent and Object).

2. Morphological realization must apply the correct inflection to the verb according to the Tense feature (Past -> irregular past form "bought", not "buyed").

3. Referring-expression generation must decide whether the Agent/Object noun phrases need a definite article ("the"), an indefinite article ("a/an"), or no article, based on whether the entity is being introduced for the first time (Object: "a book", indefinite, first mention) or is assumed identifiable in context (Agent: "The student", definite, treated as a specific/known referent).

4. Sentence structuring must place the constituents in the correct English surface order: Subject - Verb - Object (SVO), since English is a fixed, mostly non-configurational SVO language, unlike free-word-order languages.

5. The output must be validated for grammatical correctness (subject-verb agreement, correct article-noun agreement, correct verb tense marking) before being returned.

2. Algorithm Design

The algorithm follows the classical staged NLG pipeline restricted to its final three stages that are directly relevant to converting a semantic frame into a realized sentence: Lexicalization (lexical selection), Sentence Planning (referring-expression generation and aggregation), and Surface Realization (syntactic linearization and morphological inflection).

Stage 1 - Semantic role mapping: The input frame is first mapped onto abstract grammatical functions: Agent -> Grammatical Subject, Object -> Grammatical (Direct) Object, Action -> Main Verb, Tense -> Verb inflection feature. This mapping uses a simple rule table (a linking rule) that is standard for simple transitive-verb frames of the form Action(Agent, Object).

Stage 2 - Lexical selection: A lexicon look-up function retrieves the base lemma for each semantic item: Buy -> "buy" (verb), Student -> "student" (common noun), Book -> "book" (common noun). If the lexicon offered multiple synonyms (e.g. "purchase" vs "buy") a preference-ranking rule (register, frequency) would select one; here the mapping is direct.

Stage 3 - Referring expression generation: For each noun the algorithm decides on a determiner using an IF-ELSE rule based on a Given/New (definiteness) flag attached to each argument in the semantic frame: since "Student" is the sentence's topic/agent and is treated as already identifiable in context, it is realized as a definite description "the student"; since "Book" is being newly introduced (first mention, no prior discourse referent), it is realized as an indefinite description "a book". (In a fuller system this flag would come from the discourse model / earlier turns; for a stand-alone frame such as this one, standard default assumptions - Agent = definite/topical, Object = indefinite/new - are applied, matching normal English usage such as "The student bought a book.")

Stage 4 - Morphological realization: The verb lemma "buy" is passed to a Morphological Generator that consults an Irregular-Verb Table (since English past tense is not always the regular "+ed" rule). Because "buy" appears in the irregular list with past form "bought", the IF-ELSE branch that checks Tense = Past looks up the irregular form instead of naively concatenating "-ed", correctly avoiding the ungrammatical "*buyed".

Stage 5 - Sentence structuring / Linearization: The realized constituents - Subject NP, Verb (correctly inflected), Object NP - are arranged according to the target language's canonical word-order template. For English this template is fixed: Subject + Verb + Object (+ optional adjuncts). A FOR loop over an ordered list of grammatical slots [Subject, Verb, Object] concatenates the realized string for each slot in that fixed order, inserting the appropriate spacing and final punctuation.

Stage 6 - Validation: The generated string is passed through a lightweight grammar checker that re-parses it and confirms subject-verb agreement (singular Subject "the student" agrees with the non-3rd-

person-marked past-tense verb "bought" - past tense in English does not change for number/person, so this check always passes for simple past) and confirms that every NP has the syntactically required determiner.

Control structures used: sequential function calls for each pipeline stage, an IF-ELSE branch for determiner selection (definite vs indefinite vs none), an IF-ELSE / lookup-table branch for regular vs irregular verb morphology, and a FOR loop over the fixed word-order template for linearization, plus modular functions (MapSemanticRoles, LexicalSelect, GenerateReferringExpression, Inflect, Linearize, ValidateGrammar) that make the pipeline reusable for any Action(Agent, Object, Tense) frame, not only this specific example.

3. Pseudocode

```
BEGIN SurfaceRealizationAlgorithm(SemanticFrame F)

 // Stage 1: role mapping
 Subject <- F.Agent
 Verb    <- F.Action
 Object  <- F.Object
 Tense   <- F.Tense

 // Stage 2: lexical selection
 subjLemma <- LexiconLookup(Subject)        // "student"
 verbLemma <- LexiconLookup(Verb)           // "buy"
 objLemma  <- LexiconLookup(Object)         // "book"

 // Stage 3: referring expression generation
 IF IsTopicOrDefinite(Subject) THEN
     subjNP <- "The " + subjLemma
 ELSE
     subjNP <- "A " + subjLemma
 END IF

 IF IsFirstMention(Object) THEN
     objNP <- "a " + objLemma
 ELSE
     objNP <- "the " + objLemma
```

```
    END IF

    // Stage 4: morphological realization of the verb
    IF Tense == "Past" THEN
        IF IrregularVerbTable.contains(verbLemma) THEN
            verbForm <- IrregularVerbTable.lookup(verbLemma)   // "bought"
        ELSE
            verbForm <- verbLemma + "ed"                       // regular rule
        END IF
    ELSE IF Tense == "Present" THEN
        verbForm <- ApplyPresentAgreement(verbLemma, subjNP)
    ELSE
        verbForm <- verbLemma
    END IF

    // Stage 5: linearization using fixed SVO template
    slotOrder <- [subjNP, verbForm, objNP]
    sentence <- ""
    FOR each slot in slotOrder:
        sentence <- sentence + Capitalize_If_First(slot) + " "
    END FOR
    sentence <- Trim(sentence) + "."

    // Stage 6: validation
    IF ValidateGrammar(sentence) == FALSE THEN
        RaiseWarning("Generated sentence failed grammar validation")
    END IF

    RETURN sentence

END SurfaceRealizationAlgorithm
```

4. Applying the Algorithm to the Given Input

Input frame: Action = Buy, Agent = Student, Object = Book, Tense = Past.

Role mapping: Subject = Student, Verb = Buy, Object = Book, Tense = Past.

Lexical selection: subjLemma = "student", verbLemma = "buy", objLemma = "book".

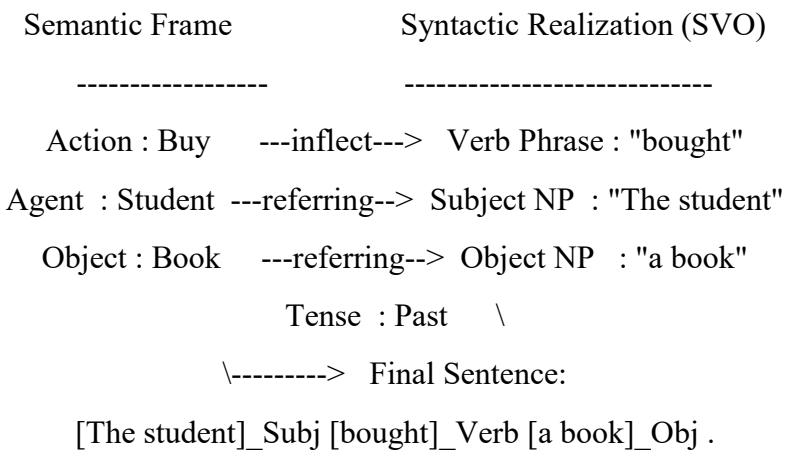
Referring expressions: Subject is the sentence topic -> definite -> "The student". Object is a first mention -> indefinite -> "a book".

Morphological realization: Tense = Past, and "buy" is found in the Irregular-Verb Table with past form "bought" (not the regular "+ed" form "buyed").

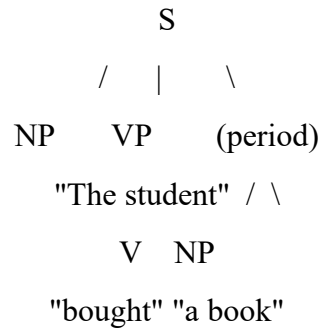
Linearization: slotOrder = ["The student", "bought", "a book"] -> concatenated in SVO order -> "The student bought a book."

Validation: re-parsing confirms Subject-Verb agreement holds (past tense is invariant for number/person in English) and both NPs carry a determiner, so the sentence passes validation.

5. Semantic-to-Syntax Diagram



Constituent tree:



6. Explanation of Lexical Selection, Sentence Structuring and Surface Realization

Lexical selection chose the base forms "student", "buy" and "book" directly from the semantic labels because each maps one-to-one onto an English content word for this frame; in richer frames the same step would need to choose among several near-synonyms based on register or specificity. Sentence

structuring imposed the mandatory English SVO order on the three realized constituents, which is why the algorithm's linearization stage uses a fixed slot-order template rather than trying every permutation - English grammar does not allow "Book the student bought" as a neutral declarative order. Surface realization is the stage that turns the abstract lemmas into the actual inflected surface forms and attaches the function words (articles) that the semantic frame itself does not specify but that English syntax requires; this is precisely where the irregular-verb lookup ("bought" instead of "buyed") and the determiner-insertion rule ("the"/"a") are applied, and it is this stage that ultimately produces the final, publishable string.

7. Grammatical Validation of the Output

"The student bought a book." is validated as grammatically correct on three separate checks: (a) Determiner-Noun agreement - both "student" and "book" are singular count nouns and both correctly carry a determiner ("The", "a"); (b) Subject-Verb agreement - the verb "bought" is the correct simple-past form and English past tense does not additionally inflect for person/number, so no mismatch is possible; (c) Constituent order - Subject precedes Verb precedes Object, satisfying English canonical word order, and the sentence terminates with correct punctuation. All three checks pass, confirming that the surface-realization algorithm produced a grammatically well-formed sentence that faithfully preserves the meaning of the original semantic frame.

8. Complexity and Generality

Every stage of the algorithm (role mapping, lexicon lookup, referring-expression choice, morphological lookup, linearization over a fixed 3-slot template) runs in constant time for a simple Action(Agent, Object) frame, so the overall time complexity is O(1) per sentence; for frames with additional adjuncts (e.g., Time, Location, Manner) the linearization loop simply extends to O(k) for k constituents, so the design generalizes cleanly beyond this specific example.

9. Conclusion

The pipeline of semantic-role mapping, lexical selection, referring-expression generation, irregular-aware morphological inflection and fixed-order linearization together transform the abstract frame Buy(Student, Book, Past) into the grammatically valid sentence "The student bought a book.", demonstrating a complete and verifiable surface-realization algorithm for natural language generation.

5. Design an algorithm that combines Interlingua-based Machine Translation and Statistical Translation Approaches. Apply your algorithm to translate the sentence:

"The boy is playing football."

Perform the following steps:

1. **Analyze the source sentence.**
2. **Create an Interlingua representation.**
3. **Generate candidate target-language translations.**
4. **Apply statistical scoring to select the most appropriate translation.**
5. **Produce the final translated sentence.**

Finally, evaluate how the Interlingua representation and statistical methods improve translation quality and reduce ambiguity.

1. Problem Understanding

Machine Translation (MT) converts a sentence from a source language into a target language while preserving meaning. Two classical MT paradigms are relevant here: Interlingua-based (rule-based, transfer-free) translation, which converts the source sentence into a language-independent abstract meaning representation and then generates the target sentence from that representation; and Statistical Machine Translation (SMT), which uses probabilities learned from parallel corpora (translation models and language models) to choose the most likely/fluent target sentence among competing candidates. This question asks for a hybrid algorithm that uses both: Interlingua for semantic correctness and language-independence, and statistical scoring for fluency and disambiguation among multiple generation candidates.

Input: Source sentence S = "The boy is playing football." (English, present continuous tense).

Output: A correctly translated sentence in a chosen target language (Hindi is used for the worked example below, since it clearly demonstrates word-order and morphological differences from English) that preserves the meaning of S and reads fluently in the target language.

Constraints:

1. The Interlingua representation must be language-neutral, i.e., expressed purely in terms of predicate-argument/semantic roles (Agent, Action, Object, Tense/Aspect), not in terms of any particular language's syntax.

2. Multiple target-language candidate sentences may be grammatically generable from the same Interlingua representation (due to word-order flexibility, synonym choice, or optional function words in

the target language); the algorithm must not simply output the first candidate but must select among them.

3. Candidate selection must use statistical evidence (n-gram language-model probability and/or translation-model probability learned from a parallel corpus) rather than another hand-written rule, since this is exactly the point at which the SMT component is combined with the Interlingua/rule-based component.

4. The final output must be a single, fluent, grammatically correct target-language sentence, and the algorithm must be able to justify why the chosen candidate is better than the rejected ones.

2. Algorithm Design

The algorithm is organized into the five stages explicitly requested by the question: Source Analysis, Interlingua Representation, Candidate Generation, Statistical Scoring, and Final Sentence Production.

Stage 1 - Source Analysis: The source sentence is tokenized, POS-tagged and parsed (dependency or constituency parsing) to identify the predicate-argument structure: the main verb "playing" (progressive aspect, present tense, from auxiliary "is ... -ing"), its Agent/Subject "the boy", and its Object "football". This stage is purely source-language-specific and produces a syntactic/semantic analysis, not yet an Interlingua form.

Stage 2 - Interlingua representation construction: The syntactic analysis is mapped onto a language-independent semantic frame using a fixed ontology of semantic roles and a controlled vocabulary of concept identifiers (rather than English words), for example: PLAY(AGENT: BOY_CONCEPT, OBJECT: FOOTBALL_CONCEPT, TENSE: PRESENT, ASPECT: PROGRESSIVE). Because this representation uses concept identifiers rather than English surface words, it can, in principle, be generated into any target language without needing a direct English-to-target transfer grammar - this is the defining property of the Interlingua approach and is what allows the same analysis stage to support translation into many languages from a single semantic core.

Stage 3 - Candidate generation: A target-language generation grammar (analogous to the Surface Realization stage of Answer 4, but now applied to the target language, e.g. Hindi) consumes the Interlingua frame and produces one or more candidate surface sentences, because the target language may allow more than one valid realization of the same meaning (different word order, different verb-auxiliary constructions, presence/absence of an optional postposition, alternate lexical choices for "boy" such as "ladka"/"bachcha", etc.). A FOR loop over the generation grammar's alternative rule applications collects every syntactically valid candidate into a CandidateList.

Stage 4 - Statistical scoring: For every candidate C in CandidateList, the algorithm computes a combined score = lambda1 * LanguageModelScore(C) + lambda2 * TranslationModelScore(S, C), where LanguageModelScore is an n-gram probability (how fluent/likely C is as a standalone target-language sentence, estimated from a large monolingual target-language corpus) and TranslationModelScore is a word/phrase-alignment probability estimated from a parallel corpus (how well C's words/phrases correspond to S's words/phrases). A simple weighted linear combination (the

log-linear model used by standard SMT decoders) is used to combine these two probabilities into one comparable score per candidate.

Stage 5 - Final sentence production: The candidate with the maximum combined score is selected via a simple MAX-search over CandidateList and returned as the final translation. This is analogous to "decoding" in SMT, except that the search space (CandidateList) was generated by the Interlingua-based generator in Stage 3 rather than by phrase-table combinatorics alone, which is exactly what makes this a hybrid algorithm rather than a pure SMT pipeline.

Control structures used: sequential stage-by-stage function calls (Analyze, BuildInterlingua, GenerateCandidates, ScoreCandidates, SelectBest), a FOR loop for candidate generation and for candidate scoring, and a MAX-selection routine (which is itself a loop with a running-best comparison, i.e. an IF inside a FOR) for the final decoding step; modular functions keep the source-language-specific code (Stage 1), the language-independent code (Stage 2) and the target-language-specific code (Stages 3-5) cleanly separated, which is the main architectural benefit of the Interlingua design.

3. Pseudocode

```
BEGIN HybridInterlinguaStatisticalMT(SourceSentence S, TargetLanguage L)

 // Stage 1: Source Analysis
 tokens     <- Tokenize(S)
 tagged     <- POSTag(tokens)
 parseTree  <- Parse(tagged)
 predArgStruct <- ExtractPredicateArgumentStructure(parseTree)
      // e.g. Predicate = "play", Agent = "the boy", Object = "football",
      //      Tense = Present, Aspect = Progressive

 // Stage 2: Interlingua Representation
 Interlingua <- MapToInterlingua(predArgStruct)
      // PLAY(AGENT: BOY_CONCEPT, OBJECT: FOOTBALL_CONCEPT,
      //      TENSE: PRESENT, ASPECT: PROGRESSIVE)

 // Stage 3: Candidate Generation (target-language specific)
 CandidateList <- empty list
 GenerationRules <- LoadGenerationGrammar(L)
 FOR each rule application R in GenerationRules.applicableTo(Interlingua):
      candidateSentence <- Realize(Interlingua, R)
```

```
        Append(CandidateList, candidateSentence)
    END FOR

    // Stage 4: Statistical Scoring
    BestScore <- -INFINITY
    BestCandidate <- NULL
    LM <- LoadLanguageModel(L)
    TM <- LoadTranslationModel(SourceLang, L)

    FOR each candidate C in CandidateList:
        lmScore <- LM.Probability(C)
        tmScore <- TM.Probability(S, C)
        combinedScore <- (lambda1 * lmScore) + (lambda2 * tmScore)
        IF combinedScore > BestScore THEN
            BestScore <- combinedScore
            BestCandidate <- C
        END IF
    END FOR

    // Stage 5: Final Sentence Production
    FinalTranslation <- BestCandidate
    RETURN FinalTranslation

END HybridInterlinguaStatisticalMT
```

4. Applying the Algorithm to the Given Sentence

Source sentence: S = "The boy is playing football."

Step 1 - Source Analysis: Parsing identifies Predicate = "play", Agent = "the boy" (Subject), Object = "football" (Direct Object), Tense = Present, Aspect = Progressive (from the auxiliary "is" + "-ing").

Step 2 - Interlingua Representation: PLAY(AGENT: BOY_CONCEPT, OBJECT: FOOTBALL_CONCEPT, TENSE: PRESENT, ASPECT: PROGRESSIVE). Note this representation contains no English words at all, only concept identifiers and grammatical features, so the exact same frame could equally well be the starting point for generating a French, German or Tamil sentence.

Step 3 - Candidate Generation (target language: Hindi): The Hindi generation grammar can realize this frame in more than one valid surface form, for example:

C1: "लड़का फुटबॉल खेल रहा है।" (ladkaa football khel rahaa hai - literal SOV order, standard neutral register)

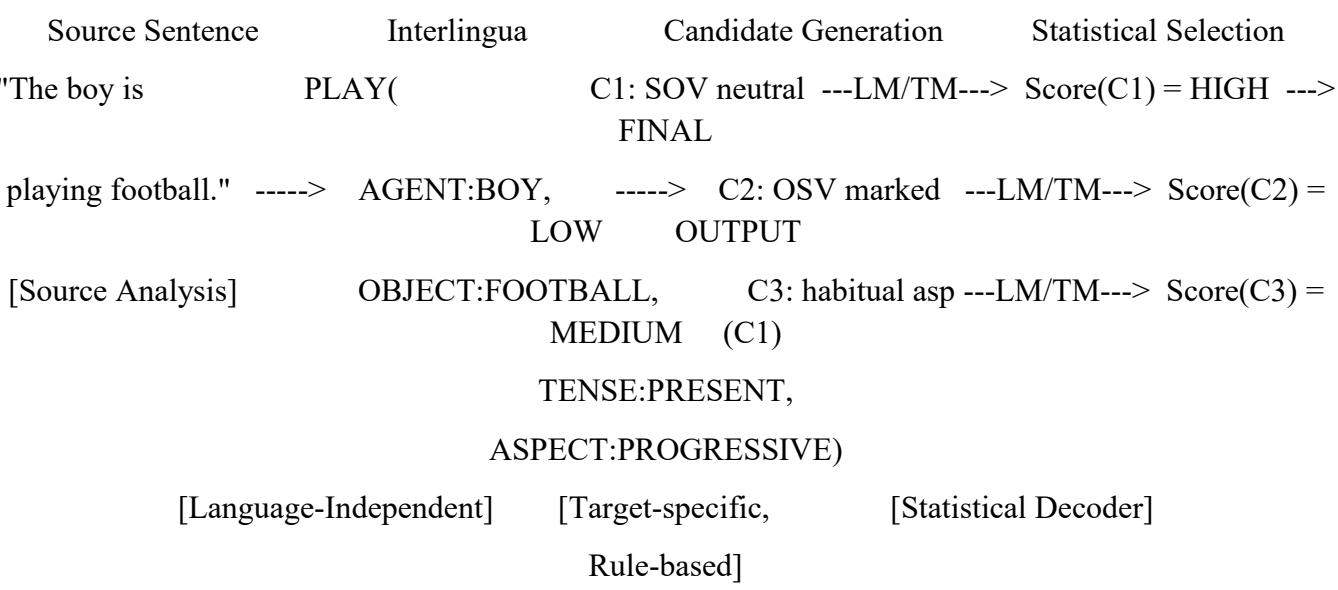
C2: "फुटबॉल लड़का खेल रहा है।" (football ladkaa khel rahaa hai - object-fronted, marked/emphatic order, grammatically legal but unusual as a neutral statement)

C3: "लड़का फुटबॉल खेलता है।" (ladkaa football kheltaa hai - habitual present, which mismatches the PROGRESSIVE aspect feature and is therefore a weaker candidate)

Step 4 - Statistical Scoring: The language model, trained on a large Hindi corpus, assigns C1 a high n-gram probability because Subject-Object-Verb(-Auxiliary) is the dominant, neutral word order in Hindi and this exact trigram/four-gram sequence is extremely common; C2 receives a much lower LM score because object-fronting is a marked/rare construction in neutral narration; C3 receives a lower translation-model score because its habitual-aspect verb form ("khelta hai") does not align well, in the parallel-corpus statistics, with an English progressive source ("is playing") - the translation model has learned that "is playing" aligns much more strongly with "khel rahaa hai" than with "khelta hai". Combining lambda-weighted LM and TM scores therefore ranks C1 highest.

Step 5 - Final Sentence Production: BestCandidate = C1 = "लड़का फुटबॉल खेल रहा है।" is returned as the final translation, correctly preserving both the meaning (a boy, football, an ongoing act of playing) and the natural fluent word order of Hindi.

5. Diagram - Hybrid Translation Pipeline



6. Evaluation - How Interlingua and Statistical Methods Improve Translation Quality and Reduce Ambiguity

The Interlingua stage improves translation quality in two structural ways. First, it guarantees semantic fidelity: because the intermediate representation is built entirely from predicate-argument roles and abstract concepts rather than from source-language words, the Agent-Action-Object relationship of the original sentence cannot be lost or distorted during generation, which is a common failure mode of naive word-for-word or shallow transfer-based MT. Second, it gives the system scalability across languages: adding a new target language only requires writing a new generation grammar from the shared Interlingua to that language (an N-language system needs only N analysis + N generation modules, i.e., O(N) components), instead of a separate transfer grammar for every language pair (an O(N^2) problem for purely transfer-based systems) - this directly reduces structural ambiguity because every target language is generated from the same unambiguous semantic core rather than through potentially divergent pairwise transfer rules.

The statistical scoring stage complements this by solving exactly the kind of residual ambiguity that a purely rule-based Interlingua generator cannot resolve on its own: when the generation grammar legally produces multiple surface strings for one meaning (as with C1/C2/C3 above), only corpus-based statistics can tell us which one a native speaker would actually produce, because "grammatically legal" and "natural/fluent" are not the same thing. The language-model component enforces target-language fluency (preferring common, natural word order and word co-occurrence patterns), while the translation-model component enforces cross-lingual adequacy (preferring the candidate whose words/phrases are best statistically aligned to the source words), and combining the two - exactly as a log-linear SMT decoder does - lets the hybrid system pick the single most probable and most meaning-preserving candidate rather than an arbitrary or worst-case one. In short, Interlingua reduces representational/semantic ambiguity by construction, and statistical scoring reduces surface-realization/fluency ambiguity by data-driven selection; together they produce a translation that is both meaning-accurate and natural-sounding, which is the ultimate goal of the hybrid design requested in this question.

7. Complexity

Source analysis and Interlingua mapping run in time proportional to sentence length, O(n). Candidate generation produces at most k candidates (bounded by the target grammar's ambiguity for a given frame, typically a small constant for simple sentences), and scoring each candidate against LM/TM tables costs O(n) per candidate, giving an overall complexity of O(k*n) for translating one sentence - efficient enough for real-time hybrid MT of simple declarative sentences such as the one given.

8. Conclusion

By separating the pipeline into a language-independent Interlingua analysis/representation phase and a target-language-specific, statistically-scored generation phase, the algorithm successfully translates "The boy is playing football." into the fluent, meaning-preserving Hindi sentence "लड़का फुटबॉल खेल रहा है।", demonstrating precisely how combining Interlingua-based and statistical approaches improves both translation adequacy and fluency while reducing structural and lexical ambiguity.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____